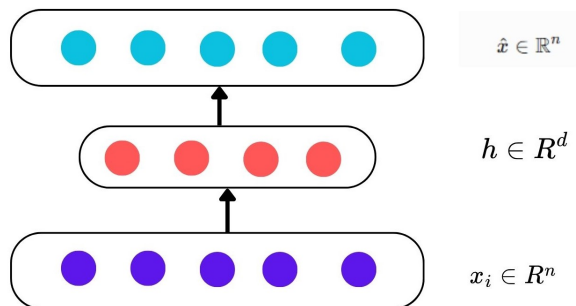


1 Autoencoder

An autoencoder is a special type of feed-forward neural network.



1. Encoder

The encoder maps the input x_i into a hidden representation h .

$$x_i \in \mathbb{R}^p$$

$$\Downarrow W$$

$$h \in \mathbb{R}^d$$

$$\Downarrow W'$$

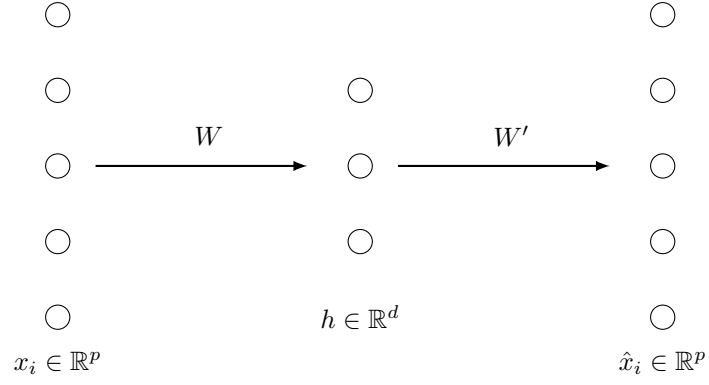
$$\hat{x}_i \in \mathbb{R}^p$$

The hidden representation is

$$h = g(Wx_i + b)$$

where

- W : encoder weight matrix,
- b : bias vector,
- $g(\cdot)$: activation function.



2. Decoder

The decoder reconstructs the original input from the hidden representation.

$$\hat{x}_i = f(W'h + c)$$

where

- W' : decoder weights,
- c : decoder bias,
- $f(\cdot)$: decoder activation.

3. The model is trained to minimize a loss function so that

$$\hat{x}_i \approx x_i.$$

4. Assume

$$\dim(h) \leq \dim(x_i).$$

If we are still able to reconstruct the input accurately, then the hidden representation h is a nearly loss-free encoding of x_i .

It captures the important characteristics of the input.

1.1 Undercomplete and Overcomplete Autoencoder

If

$$\dim(h) < \dim(x_i)$$

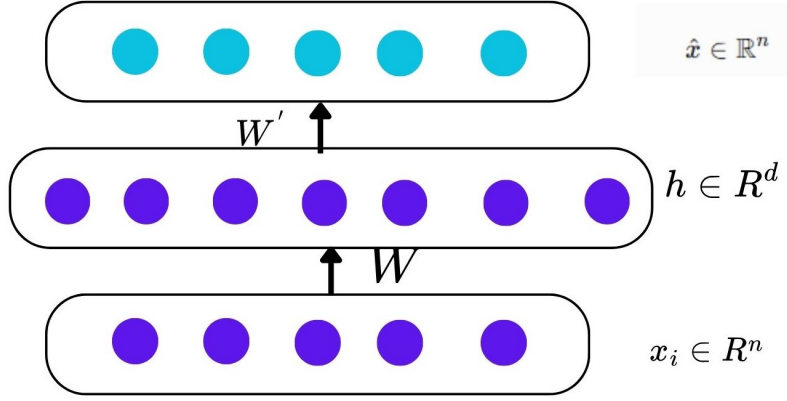
the autoencoder is called an

$$\text{Undercomplete Autoencoder.}$$

Otherwise,

$$\dim(h) \geq \dim(x_i)$$

it becomes an



Overcomplete Autoencoder.

For an overcomplete autoencoder,

$$\dim(h) \geq \dim(x_i),$$

the network can learn the trivial identity mapping,

$$x_i \longrightarrow h \longrightarrow \hat{x}_i,$$

simply by copying the input into the hidden layer and copying it back.

Such identity encoding is generally useless because it does not reveal the important characteristics of the data.

Hence,

$$\dim(h) \geq \dim(x_i)$$

corresponds to an

Overcomplete Autoencoder.

Such encoders can easily memorize the training data.

Therefore, additional constraints or regularization are required to prevent the network from learning the identity function.

2 Road Ahead: Autoencoder Design

To design an effective autoencoder, two important choices must be made:

- Choice of the encoder and decoder functions,

$$g(\cdot), \quad f(\cdot),$$

- Choice of the loss (objective) function.

We consider two categories of input data:

1. Binary-valued inputs
2. Real-valued inputs

Case 1: Binary Inputs

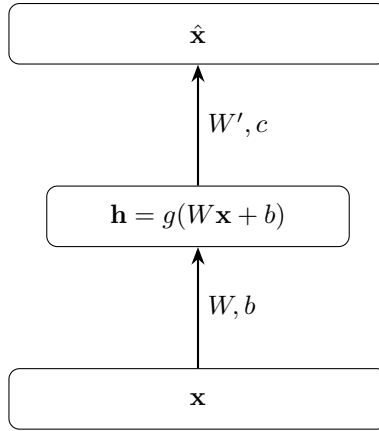
Suppose every input feature is binary,

$$x_{ij} \in \{0, 1\}.$$

The encoder generates a hidden representation

$$\mathbf{h} = g(W\mathbf{x} + b),$$

and the decoder reconstructs the input.



Possible decoder choices are

$$\hat{x}_i = \tanh(W'h + c),$$

or

$$\hat{x}_i = W'h + c,$$

or

$$\hat{x}_i = \sigma(W'h + c),$$

where

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

is the logistic sigmoid.

Since the reconstructed output must lie between 0 and 1,

$$0 \leq \hat{x}_i \leq 1,$$

the sigmoid decoder is the natural choice.

Case 2: Real-Valued Inputs

Suppose the input consists of real-valued vectors,

$$x_i \in \mathbb{R}^d.$$

The decoder may be chosen as

$$\hat{x}_i = \tanh(W'h + c),$$

or

$$\hat{x}_i = W'h + c,$$

or

$$\hat{x}_i = \sigma(W'h + c).$$

Since the sigmoid function restricts the output to

$$(0, 1),$$

it is generally unsuitable for reconstructing unrestricted real-valued data.

Hence the linear decoder

$$\hat{x} = W'h + c$$

is generally preferred.

3 Choice of Loss Function

The objective of an autoencoder is to reconstruct the input as accurately as possible.

That is,

$$\hat{x}_i \approx x_i.$$

The reconstruction error is measured using the Mean Squared Error (MSE)

$$J(W, W', b, c) = \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^d (\hat{x}_{ij} - x_{ij})^2$$

or equivalently,

$$J = \frac{1}{m} \sum_{i=1}^m (\hat{x}_i - x_i)^T (\hat{x}_i - x_i)$$

where

$$\hat{X} = \begin{bmatrix} \hat{x}_1^T \\ \hat{x}_2^T \\ \vdots \\ \hat{x}_m^T \end{bmatrix}, \quad X = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_m^T \end{bmatrix}.$$

4 Training the Autoencoder

Training proceeds exactly like a standard feedforward neural network using backpropagation.

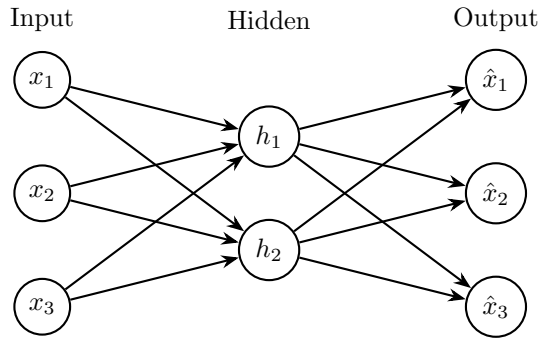
The learnable parameters are

$$\theta = \{W, W', b, c\}.$$

The required gradients are

$$\frac{\partial J}{\partial W}, \quad \frac{\partial J}{\partial W'}, \quad \frac{\partial J}{\partial b}, \quad \frac{\partial J}{\partial c}.$$

5 Neural Network Representation



The forward computation is

$$h = g(Wx + b)$$

followed by

$$\hat{x} = f(W'h + c)$$

6 Backpropagation

The reconstruction loss is

$$J = (\hat{x} - x)^T (\hat{x} - x).$$

Using the chain rule,

$$\frac{\partial J}{\partial W} = \frac{\partial J}{\partial a} \frac{\partial a}{\partial W}$$

where a denotes the pre-activation of a neuron.

Similarly,

$$\frac{\partial J}{\partial W'} = \frac{\partial J}{\partial \hat{x}} \frac{\partial \hat{x}}{\partial W'},$$

$$\frac{\partial J}{\partial b} = \frac{\partial J}{\partial a} \frac{\partial a}{\partial b},$$

and

$$\frac{\partial J}{\partial c} = \frac{\partial J}{\partial \hat{x}} \frac{\partial \hat{x}}{\partial c}.$$

The gradients are propagated backward layer by layer until every parameter has been updated.

7 Backpropagation for Mean Squared Error Loss

Recall that the reconstruction loss for a single training example is

$$\mathcal{L}^{(\theta)} = (\hat{\mathbf{x}}_i - \mathbf{x}_i)^T (\hat{\mathbf{x}}_i - \mathbf{x}_i).$$

Our objective is to compute

$$\frac{\partial \mathcal{L}^{(\theta)}}{\partial W},$$

using the chain rule.

The derivative can be expanded as

$$\boxed{\frac{\partial \mathcal{L}^{(\theta)}}{\partial W} = \frac{\partial \mathcal{L}^{(\theta)}}{\partial h} \frac{\partial h}{\partial a} \frac{\partial a}{\partial W}}$$

where

$$a = Wx + b.$$

For a multi-layer network,

$$\boxed{\frac{\partial \mathcal{L}^{(\theta)}}{\partial W} = \frac{\partial \mathcal{L}^{(\theta)}}{\partial h_2} \frac{\partial h_2}{\partial a_2} \frac{\partial a_2}{\partial h_1} \frac{\partial h_1}{\partial a_1} \frac{\partial a_1}{\partial W}}$$

Thus, we first need to compute

$$\frac{\partial \mathcal{L}^{(\theta)}}{\partial h_2}.$$

Since

$$h_2 = \hat{\mathbf{x}}_i,$$

we obtain

$$\frac{\partial \mathcal{L}^{(\theta)}}{\partial h_2} = \frac{\partial \mathcal{L}^{(\theta)}}{\partial \hat{\mathbf{x}}_i}.$$

Substituting the loss,

$$= \nabla_{\hat{\mathbf{x}}_i} [(\hat{\mathbf{x}}_i - \mathbf{x}_i)^T (\hat{\mathbf{x}}_i - \mathbf{x}_i)] .$$

Since the derivative is taken with respect to a vector,

$$\nabla_{\hat{\mathbf{x}}_i} (\mathbf{z}^T \mathbf{z}) = 2\mathbf{z},$$

where

$$\mathbf{z} = \hat{\mathbf{x}}_i - \mathbf{x}_i .$$

Hence,

$$\boxed{\frac{\partial \mathcal{L}^{(\theta)}}{\partial h_2} = 2(\hat{\mathbf{x}}_i - \mathbf{x}_i)}$$

which is a column vector.

Case 2 : Binary Inputs

Suppose the inputs are binary.

The decoder uses the logistic (sigmoid) activation,

$$\sigma(z) = \frac{1}{1 + e^{-z}} .$$

Applying the sigmoid element-wise,

$$\sigma \left(\begin{bmatrix} a_1 \\ a_2 \\ \vdots \\ a_n \end{bmatrix} \right) = \begin{bmatrix} \sigma(a_1) \\ \sigma(a_2) \\ \vdots \\ \sigma(a_n) \end{bmatrix} .$$

The sigmoid output lies between

$$0 < \sigma(z) < 1,$$

so each output can be interpreted as a probability.

For example,

$$\begin{bmatrix} 0.20 \\ 0.55 \\ 0.30 \\ 0.99 \end{bmatrix}$$

can be interpreted as the probabilities of the corresponding binary outputs.

However, these values are **not** directly binary (0 or 1).

Instead, they represent confidence or probability estimates.

The sigmoid activation is therefore a natural choice for binary reconstruction problems because it produces outputs in the interval

$$(0, 1),$$

which can be interpreted probabilistically and are suitable for binary targets.

8 Cross-Entropy Loss for Binary Inputs

For binary inputs, although the Mean Squared Error (MSE) can be used, it is generally **not** the best choice.

A much better alternative is the **Binary Cross-Entropy (BCE)** loss.

Assume a single input vector

$$\mathbf{x} = [x_1, x_2, \dots, x_n]^T,$$

where

$$x_j \in \{0, 1\}.$$

The decoder outputs

$$\hat{\mathbf{x}} = [\hat{x}_1, \hat{x}_2, \dots, \hat{x}_n]^T,$$

where

$$0 < \hat{x}_j < 1.$$

Each output can therefore be interpreted as the probability that the corresponding input bit equals one.

For a single training example, the Binary Cross-Entropy loss is

$$\mathcal{L} = - \sum_{j=1}^n [x_j \log(\hat{x}_j) + (1 - x_j) \log(1 - \hat{x}_j)]$$

This loss becomes minimum when

$$\hat{x}_j = x_j,$$

for every feature.

Interpretation

Suppose the true label is

$$x_j = 1.$$

Then

$$1 - x_j = 0,$$

and the loss reduces to

$$-\log(\hat{x}_j).$$

This quantity is minimized when

$$\hat{x}_j \rightarrow 1.$$

Similarly, if

$$x_j = 0,$$

then

$$1 - x_j = 1,$$

and the loss becomes

$$-\log(1 - \hat{x}_j),$$

which is minimized when

$$\hat{x}_j \rightarrow 0.$$

Hence Binary Cross-Entropy naturally pushes the reconstructed probability towards the correct binary value.

Probability Interpretation

Suppose the output neuron predicts two probabilities

$$q_0, \quad q_1,$$

such that

$$q_0 + q_1 = 1.$$

Therefore,

$$q_0 = 1 - q_1,$$

or equivalently,

$$q_1 = 1 - q_0.$$

The entropy of this binary distribution is

$$-\sum_{i=0}^1 p_i \log q_i.$$

Expanding,

$$-(p_0 \log q_0 + p_1 \log q_1).$$

For binary classification,

$$p_1 = x_j,$$

and

$$p_0 = 1 - x_j.$$

Hence,

$$\boxed{-[x_j \log q_1 + (1 - x_j) \log q_0]}$$

Since

$$q_1 = \hat{x}_j,$$

and

$$q_0 = 1 - \hat{x}_j,$$

the Binary Cross-Entropy becomes

$$\boxed{-[x_j \log(\hat{x}_j) + (1 - x_j) \log(1 - \hat{x}_j)]}$$

which is exactly the expression used for training autoencoders with binary inputs.

Why Cross-Entropy?

Unlike Mean Squared Error,

- Cross-Entropy treats decoder outputs as probabilities.
- It penalizes confident incorrect predictions heavily.
- It usually converges faster during gradient descent.
- It is the standard loss for sigmoid output neurons.

Examples

If

$$x_j = 1,$$

the loss is minimized by

$$\hat{x}_j = 1.$$

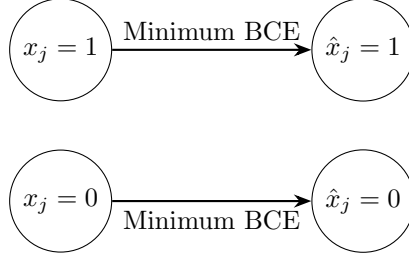
If

$$x_j = 0,$$

the loss is minimized by

$$\hat{x}_j = 0.$$

These two cases are illustrated below.



9 Applications of Autoencoders

Autoencoders have become one of the most important unsupervised learning techniques because they can automatically learn useful representations of data. Some important applications are discussed below.

9.1 1. Dimensionality Reduction

The hidden layer of an undercomplete autoencoder learns a compact representation of the input data.

Instead of storing the original high-dimensional input

$$x \in \mathbb{R}^n,$$

the encoder produces

$$h \in \mathbb{R}^k, \quad k < n.$$

This lower-dimensional representation retains most of the useful information.

Unlike Principal Component Analysis (PCA), an autoencoder can learn **non-linear** transformations.

9.2 2. Image Denoising

Suppose the input image is corrupted by random noise,

$$x = x_{\text{clean}} + \text{noise}.$$

Instead of reconstructing the noisy image, the autoencoder is trained to produce the clean image.

$$x_{\text{noisy}} \longrightarrow \text{Encoder} \longrightarrow h \longrightarrow \text{Decoder} \longrightarrow x_{\text{clean}}.$$

The loss function becomes

$$L = \|x_{\text{clean}} - \hat{x}\|^2.$$

The network therefore learns to remove unwanted disturbances.

9.3 3. Anomaly Detection

An autoencoder is trained only on normal data.

During testing,

$$e = \|x - \hat{x}\|^2$$

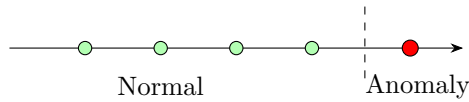
is computed.

If

$$e > \tau,$$

where τ is a threshold,
the sample is declared an anomaly.
Examples include

- Credit card fraud detection
- Industrial fault detection
- Network intrusion detection
- Medical diagnosis



9.4 4. Data Compression

Instead of storing the original data,

$$x \in \mathbb{R}^n,$$

only the latent representation

$$h \in \mathbb{R}^k, \quad k \ll n,$$

is stored.

Whenever needed,

$$\hat{x} = f(h)$$

is reconstructed.

Compression ratio is

$$\text{Compression Ratio} = \frac{n}{k}.$$

9.5 5. Feature Extraction

The encoder automatically extracts meaningful features.

Instead of manually designing features,

$$x \longrightarrow h,$$

where

$$h = g(Wx + b)$$

contains the important characteristics of the input.

These features can later be used for

- Classification
- Regression
- Clustering
- Visualization

9.6 6. Recommendation Systems

Autoencoders are widely used in recommender systems.

Suppose

$$x = \begin{bmatrix} 5 \\ 4 \\ ? \\ 1 \\ ? \end{bmatrix}$$

represents ratings given by a user.

The encoder learns latent user preferences.

The decoder predicts the missing entries.

$$\hat{x} = \begin{bmatrix} 5 \\ 4 \\ 3.8 \\ 1 \\ 4.6 \end{bmatrix}$$

Examples include

- Netflix
- Amazon
- YouTube
- Spotify

9.7 7. Image and Video Applications

Autoencoders have numerous applications in computer vision.

Some important applications include

1. Super-resolution
2. Colorization
3. Image Segmentation
4. Face Recognition
5. Pose Estimation
6. Image Restoration
7. Video Reconstruction

9.8 Summary of Applications

Table 1: Major Applications of Autoencoders

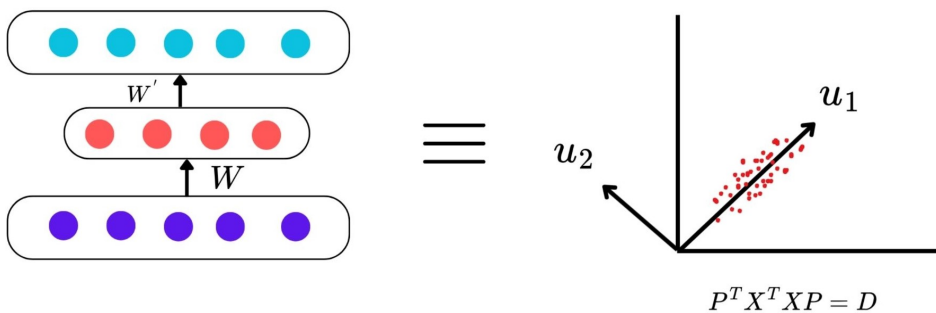
Application	Purpose
Dimensionality Reduction	Learn compact representations
Denoising	Remove unwanted noise from data
Anomaly Detection	Detect unusual samples
Compression	Reduce storage requirements
Feature Learning	Learn useful hidden representations
Recommendation Systems	Predict missing preferences
Computer Vision	Super-resolution, segmentation, face recognition, restoration

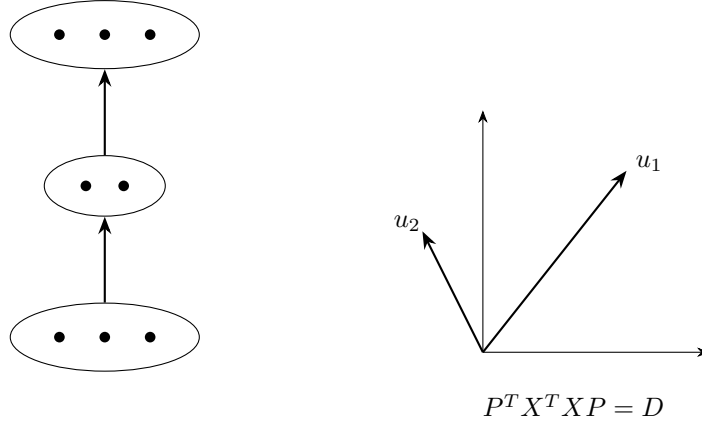
10 Key Points

- Autoencoders are unsupervised neural networks.
- They learn by reconstructing the input.
- The encoder compresses the information.
- The decoder reconstructs the original input.
- Undercomplete autoencoders perform dimensionality reduction.
- Overcomplete autoencoders require regularization.
- Binary Cross-Entropy is preferred for binary inputs.
- Mean Squared Error is preferred for continuous-valued inputs.
- Autoencoders form the basis of many modern deep learning models such as Variational Autoencoders (VAEs), Denoising Autoencoders, Sparse Autoencoders, Contractive Autoencoders, and Diffusion-based representation learning.

11 Link Between PCA and Autoencoder

We now establish the relationship between Principal Component Analysis (PCA) and Autoencoders.





We shall now show that the encoder part of an Autoencoder is equivalent to Principal Component Analysis provided the following conditions are satisfied.

1. A **linear encoder** is used.
2. A **linear decoder** is used.
3. The reconstruction loss is the squared error loss.
4. The input data are normalized (zero mean).

11.1 Normalization of the Input Data

Each feature is normalized according to

$$\hat{x}_{ij} = \frac{1}{\sqrt{m}} \left(x_{ij} - \frac{1}{m} \sum_{k=1}^m x_{kj} \right)$$

where

$$\mu_j = \frac{1}{m} \sum_{k=1}^m x_{kj}$$

is the mean of the j^{th} feature.

The quantity inside the brackets subtracts the feature mean from every sample. Hence, after normalization,

$$\sum_{i=1}^m \hat{x}_{ij} = 0, \quad j = 1, \dots, n.$$

Therefore, every feature has zero mean.

Let

$$X'$$

denote the zero-mean data matrix.

Then,

$$\hat{X} = \frac{1}{\sqrt{m}} X'.$$

Consequently,

$$\begin{aligned}\hat{X}^T \hat{X} &= \left(\frac{1}{\sqrt{m}} X' \right)^T \left(\frac{1}{\sqrt{m}} X' \right) \\ &= \frac{1}{m} (X')^T X' .\end{aligned}$$

Since

$$C = \frac{1}{m} (X')^T X' ,$$

we obtain

$$\hat{X}^T \hat{X} = C ,$$

which is exactly the covariance matrix used in PCA.

11.2 Linear Autoencoder Objective

Assume that both encoder and decoder are linear.

The reconstruction loss is

$$J = \frac{1}{m} \sum_{i=1}^m \sum_{j=1}^n (x_{ij} - \hat{x}_{ij})^2 .$$

Equivalently,

$$\min_{\theta} \sum_{i=1}^m \sum_{j=1}^n (x_{ij} - \hat{x}_{ij})^2 .$$

Since

$$\hat{X} = HW^T ,$$

the optimization problem can be written compactly as

$$\min_{H,W} \|X - HW^T\|_F^2 ,$$

where

$$\|A\|_F = \sqrt{\sum_{i=1}^m \sum_{j=1}^n a_{ij}^2}$$

is the Frobenius norm.

The matrices involved are

$$X = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_m \end{bmatrix} \in \mathbb{R}^{m \times n} ,$$

$$H = \begin{bmatrix} h_1 \\ h_2 \\ \vdots \\ h_m \end{bmatrix} \in \mathbb{R}^{m \times k},$$

and

$$W \in \mathbb{R}^{n \times k}.$$

Hence,

$$\hat{X} = HW^T.$$

11.3 Optimal Solution using Singular Value Decomposition (SVD)

The optimization problem obtained previously is

$$\min_{H, W} \|X - HW^T\|_F^2. \quad (1)$$

Recall the Singular Value Decomposition (SVD) of the data matrix,

$$X = U\Sigma V^T.$$

From the Eckart–Young theorem, the best rank- k approximation of X is

$$X_k = U_{(:,1:k)} \Sigma_{1:k,1:k} V_{(:,1:k)}^T.$$

Therefore, the optimal reconstruction satisfying (1) is

$$HW^T = U_{(:,1:k)} \Sigma_{1:k,1:k} V_{(:,1:k)}^T.$$

One possible choice satisfying this equality is

$$H = U_{(:,1:k)} \Sigma_{1:k,1:k},$$

and

$$W = V_{(:,1:k)}.$$

Thus,

$$HW^T = U_{(:,1:k)} \Sigma_{1:k,1:k} V_{(:,1:k)}^T.$$

Hence the linear decoder weights correspond to the first k right singular vectors of the data matrix.

11.4 Derivation of the Encoder Matrix

We now derive the encoder representation.

Recall that

$$H = XW.$$

Substituting the optimal decoder,

$$W = V_{(:,1:k)},$$

gives

$$H = XV_{(:,1:k)}.$$

To verify this using the SVD expansion,

$$X = U\Sigma V^T,$$

we write

$$\begin{aligned} H &= XV_{(:,1:k)} \\ &= U\Sigma V^T V_{(:,1:k)}. \end{aligned}$$

Since

$$V^T V = I,$$

we obtain

$$\boxed{H = U\Sigma_{(:,1:k)}}.$$

This is exactly the expression obtained from the optimal rank- k approximation.

11.5 Alternative Proof Using the Moore–Penrose Inverse

Previously, we obtained

$$H^* = U_{(:,1:k)} \Sigma_{(1:k,1:k)}.$$

The encoder weight matrix is

$$W^* = X^+ H^*,$$

where X^+ denotes the Moore–Penrose pseudoinverse.

Using

$$X^+ = (X^T X)^{-1} X^T,$$

we obtain

$$W^* = (X^T X)^{-1} X^T U_{(:,1:k)} \Sigma_{(1:k,1:k)}.$$

Now substitute the Singular Value Decomposition

$$X = U\Sigma V^T.$$

Hence,

$$= (V\Sigma^T U^T U\Sigma V^T)^{-1} (V\Sigma^T U^T) U_{(:,1:k)} \Sigma_{(1:k,1:k)}.$$

(using $X = U\Sigma V^T$)

Since

$$U^T U = I,$$

the above becomes

$$= (V\Sigma^T \Sigma V^T)^{-1} V\Sigma^T U^T U_{(:,1:k)} \Sigma_{(1:k,1:k)}.$$

Using

$$(ABC)^{-1} = C^{-1}B^{-1}A^{-1},$$

we obtain

$$= V(\Sigma^T \Sigma)^{-1} V^T V\Sigma^T U^T U_{(:,1:k)} \Sigma_{(1:k,1:k)}.$$

Since

$$V^T V = I,$$

this simplifies to

$$= V(\Sigma^T \Sigma)^{-1} \Sigma^T U^T U_{(:,1:k)} \Sigma_{(1:k,1:k)}.$$

Now observe that

$$(\Sigma^T \Sigma)^{-1} \Sigma^T = \Sigma^{-1},$$

therefore,

$$= V\Sigma^{-1} U^T U_{(:,1:k)} \Sigma_{(1:k,1:k)}.$$

Since

$$U^T U_{(:,1:k)} = I_{(:,1:k)},$$

we obtain

$$= V\Sigma^{-1} I_{(:,1:k)} \Sigma_{(1:k,1:k)}.$$

Finally,

$$\Sigma^{-1} I_{(:,1:k)} \Sigma_{(1:k,1:k)} = I_{(:,1:k)},$$

which gives

$$\boxed{W^* = V_{(:,1:k)}}$$

Hence, we have shown that the optimal encoder of a linear autoencoder is

$$\boxed{H = XW^* = XV_{(:,1:k)}}.$$

Therefore, the encoder performs a linear transformation using the first k right singular vectors of X . Since the right singular vectors of X are precisely the eigenvectors of $X^T X$, the encoder weight matrix is identical to the projection matrix used in Principal Component Analysis (PCA).

Thus,

$$W^* = P = V_{(:,1:k)}.$$

Hence, the encoder of a linear autoencoder and the PCA projection matrix are exactly the same.

11.6 Connection with PCA

From Singular Value Decomposition,

$$W = V_{(:,1:k)}.$$

From PCA, the projection matrix is formed by the eigenvectors of the covariance matrix

$$C = \frac{1}{m} X^T X.$$

Since

$$X^T X = V \Sigma^2 V^T,$$

the columns of V are precisely the eigenvectors of the covariance matrix.

Therefore,

$$P = V_{(:,1:k)}.$$

Consequently,

$$W = P.$$

That is, the encoder matrix learned by a linear autoencoder is exactly the PCA projection matrix.

11.7 Final Result

We have shown that the encoder of a linear autoencoder is equivalent to PCA provided the following conditions are satisfied.

1. A linear encoder is used.
2. A linear decoder is used.
3. The reconstruction loss is the squared-error loss.
4. The input data are normalized to have zero mean.

Since the normalized data satisfy

$$\hat{x}_{ij} = \frac{1}{\sqrt{m}} \left(x_{ij} - \frac{1}{m} \sum_{k=1}^m x_{kj} \right),$$

the covariance matrix becomes

$$C = \hat{X}^T \hat{X},$$

whose eigenvectors are exactly

$$V.$$

Hence,

$$\text{Linear Autoencoder Encoder} = \text{PCA Projection Matrix.}$$

Important Result

The encoder of a linear autoencoder is equivalent to Principal Component Analysis (PCA) if

- the encoder is linear,
- the decoder is linear,
- the reconstruction loss is Mean Squared Error,
- and the input data are centered (zero mean).

Under these assumptions,

$$W = P = V_{(:,1:k)},$$

where $V_{(:,1:k)}$ contains the first k eigenvectors (principal components) of the covariance matrix.

12 Equivalence of Linear Autoencoder and PCA (Continued)

From the previous derivation, we obtained

$$W = V_{(:,1:k)},$$

where $V_{(:,1:k)}$ denotes the first k eigenvectors (principal components) of the covariance matrix.

Recall that the normalized data matrix is

$$\hat{x}_{ij} = \frac{1}{\sqrt{m}} \left(x_{ij} - \frac{1}{m} \sum_{l=1}^m x_{lj} \right),$$

which centers every feature so that its mean becomes zero.

Since

$$X = \frac{1}{\sqrt{m}} X',$$

the covariance matrix becomes

$$X^T X = \frac{1}{m} (X')^T X'.$$

The covariance matrix plays the central role in Principal Component Analysis.

Since

$$X = U\Sigma V^T,$$

the eigenvectors of

$$X^T X$$

are exactly the columns of

$$V.$$

Therefore,

$$W = V_{(:,1:k)}$$

is precisely the PCA projection matrix.

Hence, the encoder of a linear autoencoder performs the same projection as Principal Component Analysis.

12.1 Important Result

The encoder of a linear autoencoder is equivalent to PCA provided that

1. the encoder is linear,
2. the decoder is linear,
3. the reconstruction loss is Mean Squared Error,
4. the input data are centered (zero mean).

Under these assumptions,

$$W = P = V_{(:,1:k)}$$

where P is the PCA projection matrix.

13 Relationship Between PCA and Autoencoder

Figure 1 illustrates the similarity between a linear autoencoder and Principal Component Analysis.

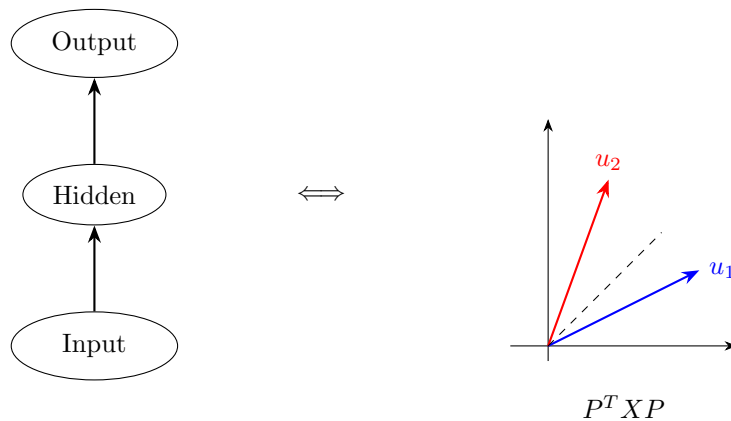


Figure 1: Connection between Linear Autoencoder and PCA

14 Summary

For a linear autoencoder,

$$H = XW,$$

where

$$W = V_{(:,1:k)}.$$

The reconstruction is

$$\hat{X} = HW^T.$$

Therefore,

$$\hat{X} = XV_{(:,1:k)}V_{(:,1:k)}^T,$$

which is exactly the rank- k approximation obtained from Principal Component Analysis. Hence,

Linear Autoencoder $\equiv$ PCA

when

- encoder is linear,
- decoder is linear,
- reconstruction loss is MSE,
- inputs are centered.

15 Advantages of Autoencoders over PCA

Although linear autoencoders reduce to PCA, practical autoencoders are generally nonlinear. Their major advantages are

- Nonlinear feature extraction.
- Deep hierarchical representations.
- Better reconstruction for complex data.
- Ability to model images, speech and text.
- Easy integration with CNNs, RNNs and Transformers.
- Can be trained using gradient descent.

Thus,

PCA is a special case of a Linear Autoencoder.

A nonlinear autoencoder extends PCA by replacing the linear encoder and decoder with nonlinear neural networks.

16 Transition to Deep Autoencoders

Instead of

$$h = f(Wx + b),$$

we may use multiple hidden layers:

$$x \rightarrow h_1 \rightarrow h_2 \rightarrow \cdots \rightarrow h_k \rightarrow \hat{x}.$$

This leads to Deep Autoencoders, which can learn highly nonlinear latent representations that are impossible to obtain using classical PCA.

17 Denoising Autoencoder

A denoising autoencoder is a special type of autoencoder in which the input data is deliberately corrupted before being given to the network. The objective of the network is to reconstruct the original uncorrupted input.

The basic idea is

$$\boxed{\text{Corrupted Input} \longrightarrow \text{Encoder} \longrightarrow \text{Decoder} \longrightarrow \text{Reconstructed Input}}$$

The important difference from a conventional autoencoder is that the network receives a corrupted version of the input but is trained to reconstruct the original input.

17.1 Basic Architecture

Let the original input be

$$x_i \in \mathbb{R}^n.$$

A corrupted version of the input is denoted by

$$\tilde{x}_i.$$

The corruption process is represented by

$$\tilde{x}_i \sim P(\tilde{x}_i \mid x_i).$$

The corrupted input is then passed through the encoder:

$$h_i = g(W\tilde{x}_i + b).$$

The decoder reconstructs the original input:

$$\hat{x}_i = f(W'h_i + c).$$

Therefore,

$$\boxed{x_i \longrightarrow \tilde{x}_i \longrightarrow h_i \longrightarrow \hat{x}_i}$$

where

- x_i is the original input,
- $\tilde{x}_i$ is the corrupted input,
- h_i is the hidden representation,
- $\hat{x}_i$ is the reconstructed output.

17.2 Denoising Autoencoder Architecture

The architecture can be represented as follows.

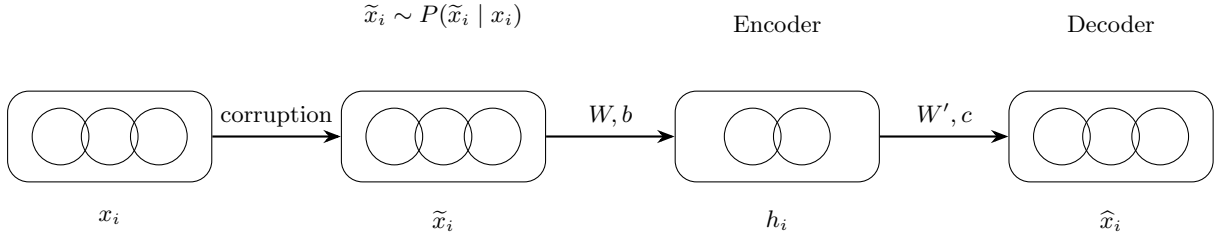


Figure 2: Architecture of a Denoising Autoencoder

17.3 Corruption Process

Consider an input vector

$$x_i = \begin{bmatrix} x_{i1} & x_{i2} & \cdots & x_{in} \end{bmatrix}^T.$$

A corrupted version of the input is generated using a corruption distribution

$$P(\tilde{x}_i | x_i)$$

The corruption process changes some components of x_i .

For example,

$$x_i = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

may be transformed into

$$\tilde{x}_i = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 \end{bmatrix}.$$

Thus, some of the original features are intentionally removed or corrupted.

17.4 Simple Corruption Model

A simple corruption process is obtained by randomly setting an input feature to zero.

For each input component,

$$x_{ij} \in \{0, 1\},$$

we can define

$$P(\tilde{x}_{ij} = 0 | x_{ij}) = q,$$

and

$$P(\tilde{x}_{ij} = x_{ij} \mid x_{ij}) = 1 - q.$$

Here,

$$q$$

is the probability of corrupting an input component.

Thus, with probability q , an input feature is corrupted, while with probability $1 - q$, it is retained.

For example, consider

$$x_i = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \end{bmatrix}.$$

After applying the corruption process, we may obtain

$$\tilde{x}_i = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 \end{bmatrix}.$$

The network receives

$$\tilde{x}_i$$

but the desired output remains

$$x_i.$$

Therefore,

$$\tilde{x}_i \neq x_i$$

but

$$\hat{x}_i \approx x_i.$$

17.5 Encoder and Decoder

The corrupted input is passed through the encoder:

$$h_i = g(W\tilde{x}_i + b)$$

where

$$W \in \mathbb{R}^{d \times n}.$$

The hidden representation is then passed through the decoder:

$$\hat{x}_i = f(W'h_i + c)$$

where

$$W' \in \mathbb{R}^{n \times d}.$$

Combining both operations gives

$$\hat{x}_i = f(W'g(W\tilde{x}_i + b) + c).$$

The important point is that the decoder is not asked to reproduce the corrupted input. Instead, it is trained to reproduce the original input:

$$\boxed{\tilde{x}_i \longrightarrow \hat{x}_i \approx x_i.}$$

17.6 Why Denoising Autoencoder?

In a conventional autoencoder, the network receives

$$x_i$$

and attempts to reconstruct

$$x_i.$$

Thus,

$$x_i \longrightarrow h_i \longrightarrow \hat{x}_i \approx x_i.$$

In a denoising autoencoder, the input is first corrupted:

$$x_i \longrightarrow \tilde{x}_i \longrightarrow h_i \longrightarrow \hat{x}_i.$$

However, the target remains the original input:

$$\boxed{\hat{x}_i \approx x_i.}$$

Therefore, the network is forced to learn the important underlying features of the data rather than simply copying the input.

17.7 Reconstruction Objective

The objective of the denoising autoencoder is to minimize the reconstruction error between the original input x_i and the reconstructed output $\hat{x}_i$.

For m training examples, the Mean Squared Error objective is

$$\boxed{\min_{\theta} \frac{1}{m} \sum_{i=1}^m \|x_i - \hat{x}_i\|_2^2}$$

where

$$\theta = \{W, b, W', c\}.$$

Since

$$\hat{x}_i = f(W'g(W\tilde{x}_i + b) + c),$$

the complete objective becomes

$$\min_{W, W', b, c} \frac{1}{m} \sum_{i=1}^m \|x_i - f(W'g(W\tilde{x}_i + b) + c)\|_2^2$$

with

$$\tilde{x}_i \sim P(\tilde{x}_i | x_i).$$

17.8 Key Idea

The key idea behind a denoising autoencoder is therefore

Corrupt the input $\rightarrow$ encode the corrupted input $\rightarrow$ reconstruct the original input.

The corruption process prevents the network from simply learning the identity function.

Instead, the network must learn robust features that allow it to recover the original data from a partially corrupted observation.

Thus,

DAE learns robust representations by reconstructing clean data from corrupted data.

17.9 Effect of Corruption

Suppose the original input is

$$x_i = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \end{bmatrix}^T.$$

After corruption,

$$\tilde{x}_i = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 \end{bmatrix}^T.$$

The encoder receives

$$h_i = g(W\tilde{x}_i + b),$$

and the decoder produces

$$\hat{x}_i = f(W'h_i + c).$$

The target is still

$$x_i = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \end{bmatrix}^T.$$

Therefore, the training objective forces

$$\hat{x}_i \rightarrow x_i$$

even though the network only receives

$$\tilde{x}_i.$$

This makes the learned representation more robust to noise and missing features.

17.10 Graphical Interpretation of the Corruption Process

The corruption process can be illustrated as

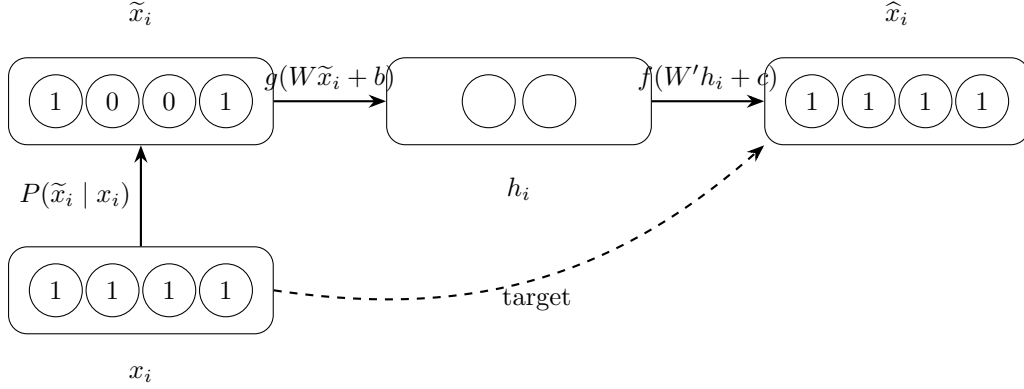


Figure 3: Denoising process: corrupt the input and reconstruct the original clean input

17.11 Important Observation

A conventional autoencoder tries to learn

$$x_i \rightarrow x_i,$$

which can lead to learning an identity mapping.

A denoising autoencoder instead learns

$$\tilde{x}_i \rightarrow x_i.$$

Therefore, the network cannot simply copy its input. It must learn the underlying structure of the data.

This results in

more robust feature representations.

18 Denoising Autoencoder

A denoising autoencoder is a special type of autoencoder in which the input data is deliberately corrupted before being given to the network. The objective of the network is to reconstruct the original uncorrupted input.

The basic idea is

Corrupted Input $\rightarrow$ Encoder $\rightarrow$ Decoder $\rightarrow$ Reconstructed Input

The important difference from a conventional autoencoder is that the network receives a corrupted version of the input but is trained to reconstruct the original input.

18.1 Basic Architecture

Let the original input be

$$x_i \in \mathbb{R}^n.$$

A corrupted version of the input is denoted by

$$\tilde{x}_i.$$

The corruption process is represented by

$$\tilde{x}_i \sim P(\tilde{x}_i | x_i).$$

The corrupted input is then passed through the encoder:

$$h_i = g(W\tilde{x}_i + b).$$

The decoder reconstructs the original input:

$$\hat{x}_i = f(W'h_i + c).$$

Therefore,

$$x_i \longrightarrow \tilde{x}_i \longrightarrow h_i \longrightarrow \hat{x}_i$$

where

- x_i is the original input,
- $\tilde{x}_i$ is the corrupted input,
- h_i is the hidden representation,
- $\hat{x}_i$ is the reconstructed output.

18.2 Denoising Autoencoder Architecture

The architecture can be represented as follows.

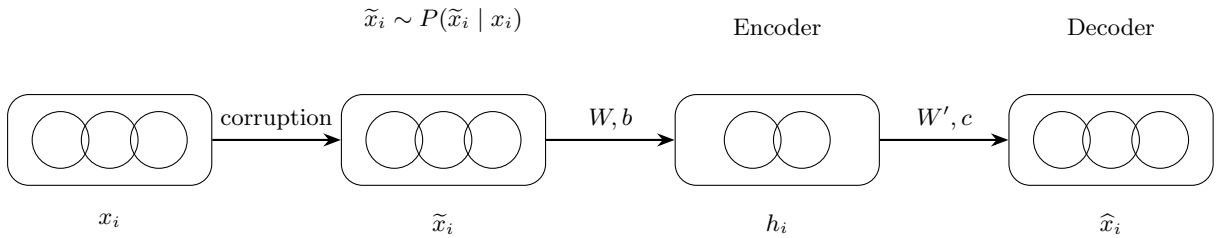


Figure 4: Architecture of a Denoising Autoencoder

18.3 Corruption Process

Consider an input vector

$$x_i = \begin{bmatrix} x_{i1} & x_{i2} & \cdots & x_{in} \end{bmatrix}^T.$$

A corrupted version of the input is generated using a corruption distribution

$$P(\tilde{x}_i | x_i)$$

The corruption process changes some components of x_i .

For example,

$$x_i = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \end{bmatrix}$$

may be transformed into

$$\tilde{x}_i = \begin{bmatrix} 1 & 0 & 1 & 0 & 1 \end{bmatrix}.$$

Thus, some of the original features are intentionally removed or corrupted.

18.4 Simple Corruption Model

A simple corruption process is obtained by randomly setting an input feature to zero.

For each input component,

$$x_{ij} \in \{0, 1\},$$

we can define

$$P(\tilde{x}_{ij} = 0 | x_{ij}) = q,$$

and

$$P(\tilde{x}_{ij} = x_{ij} | x_{ij}) = 1 - q.$$

Here,

$$q$$

is the probability of corrupting an input component.

Thus, with probability q , an input feature is corrupted, while with probability $1 - q$, it is retained.

For example, consider

$$x_i = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \end{bmatrix}.$$

After applying the corruption process, we may obtain

$$\tilde{x}_i = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 \end{bmatrix}.$$

The network receives

$$\tilde{x}_i$$

but the desired output remains

$$x_i.$$

Therefore,

$$\tilde{x}_i \neq x_i$$

but

$$\hat{x}_i \approx x_i.$$

18.5 Encoder and Decoder

The corrupted input is passed through the encoder:

$$h_i = g(W\tilde{x}_i + b)$$

where

$$W \in \mathbb{R}^{d \times n}.$$

The hidden representation is then passed through the decoder:

$$\hat{x}_i = f(W'h_i + c)$$

where

$$W' \in \mathbb{R}^{n \times d}.$$

Combining both operations gives

$$\hat{x}_i = f(W'g(W\tilde{x}_i + b) + c).$$

The important point is that the decoder is not asked to reproduce the corrupted input. Instead, it is trained to reproduce the original input:

$$\tilde{x}_i \longrightarrow \hat{x}_i \approx x_i.$$

18.6 Why Denoising Autoencoder?

In a conventional autoencoder, the network receives

$$x_i$$

and attempts to reconstruct

$$x_i.$$

Thus,

$$x_i \longrightarrow h_i \longrightarrow \hat{x}_i \approx x_i.$$

In a denoising autoencoder, the input is first corrupted:

$$x_i \longrightarrow \tilde{x}_i \longrightarrow h_i \longrightarrow \hat{x}_i.$$

However, the target remains the original input:

$$\widehat{x}_i \approx x_i.$$

Therefore, the network is forced to learn the important underlying features of the data rather than simply copying the input.

18.7 Reconstruction Objective

The objective of the denoising autoencoder is to minimize the reconstruction error between the original input x_i and the reconstructed output $\widehat{x}_i$.

For m training examples, the Mean Squared Error objective is

$$\min_{\theta} \frac{1}{m} \sum_{i=1}^m \|x_i - \widehat{x}_i\|_2^2$$

where

$$\theta = \{W, b, W', c\}.$$

Since

$$\widehat{x}_i = f(W'g(W\tilde{x}_i + b) + c),$$

the complete objective becomes

$$\min_{W, W', b, c} \frac{1}{m} \sum_{i=1}^m \|x_i - f(W'g(W\tilde{x}_i + b) + c)\|_2^2$$

with

$$\tilde{x}_i \sim P(\tilde{x}_i | x_i).$$

18.8 Key Idea

The key idea behind a denoising autoencoder is therefore

$$\text{Corrupt the input} \rightarrow \text{encode the corrupted input} \rightarrow \text{reconstruct the original input.}$$

The corruption process prevents the network from simply learning the identity function.

Instead, the network must learn robust features that allow it to recover the original data from a partially corrupted observation.

Thus,

$$\text{DAE learns robust representations by reconstructing clean data from corrupted data.}$$

18.9 Effect of Corruption

Suppose the original input is

$$x_i = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \end{bmatrix}^T.$$

After corruption,

$$\tilde{x}_i = \begin{bmatrix} 1 & 0 & 0 & 1 & 1 \end{bmatrix}^T.$$

The encoder receives

$$h_i = g(W\tilde{x}_i + b),$$

and the decoder produces

$$\hat{x}_i = f(W'h_i + c).$$

The target is still

$$x_i = \begin{bmatrix} 1 & 1 & 1 & 1 & 1 \end{bmatrix}^T.$$

Therefore, the training objective forces

$$\hat{x}_i \rightarrow x_i$$

even though the network only receives

$$\tilde{x}_i.$$

This makes the learned representation more robust to noise and missing features.

18.10 Graphical Interpretation of the Corruption Process

The corruption process can be illustrated as

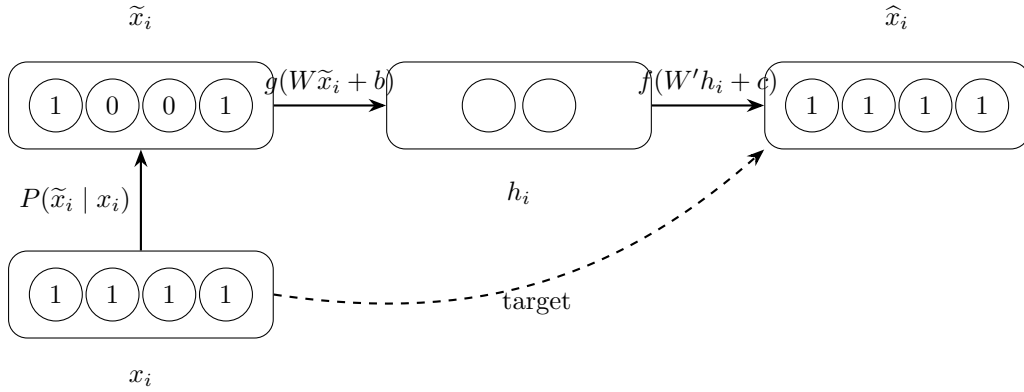


Figure 5: Denoising process: corrupt the input and reconstruct the original clean input

18.11 Important Observation

A conventional autoencoder tries to learn

$$x_i \rightarrow x_i,$$

which can lead to learning an identity mapping.

A denoising autoencoder instead learns

$$\boxed{\tilde{x}_i \rightarrow x_i.}$$

Therefore, the network cannot simply copy its input. It must learn the underlying structure of the data.

This results in

$$\boxed{\text{more robust feature representations.}}$$

19 Feature Learning using Autoencoders

For example, it will have to learn how to reconstruct a corrupted x_i correctly by relying on its interaction with different elements of x_i .

So, what is our objective is that we introduce error in the input and try to improve the noise of the autoencoder.

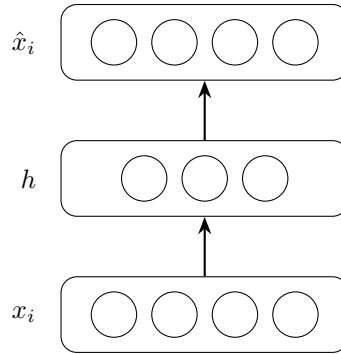


Figure 6: Autoencoder representation

$$\underbrace{\begin{bmatrix} 0 & 0 & 0 & \dots & 0 \end{bmatrix}}_{x_i}$$

For MNIST data,

$$|x_i| = 28 \times 28 = 784.$$

Therefore,

$$x_i \in \mathbb{R}^{784}.$$

Suppose the hidden representation is

$$h \in \mathbb{R}^d,$$

where

$$d < 784.$$

Thus, the autoencoder learns a lower-dimensional representation of the input data.

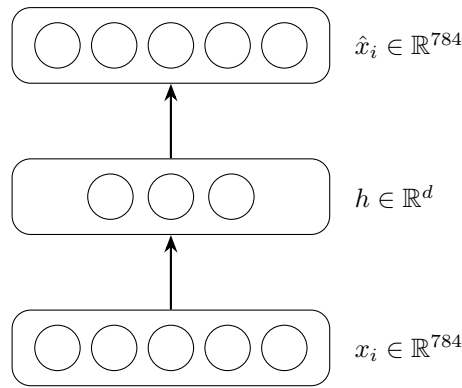


Figure 7: Dimensionality reduction using an autoencoder

19.1 Hidden Feature Representation

What happens here, we know the input data, and we have perfect feature representation which exactly captures the data.

Our original aim was to do classification.

So what we need to do?

We keep the first few layers of the autoencoder and throw away the last layer.

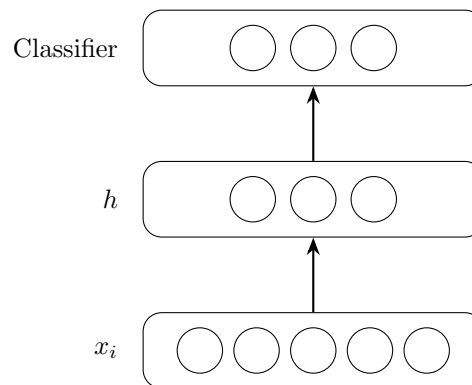


Figure 8: Using learned hidden features for classification

We can learn hidden feature value from AE, and feed the feature layer to the classifier.

$$\underbrace{\begin{bmatrix} 0 & 0 & 0 & 0 & 0 \end{bmatrix}}_{\text{input}}$$

$$\underbrace{\begin{bmatrix} 0 & 0 & 0 \end{bmatrix}}_{\text{hidden feature}}$$

For example,

$$h_i = \sigma(w_i^T x_i),$$

where w_i is the shared vector of weights connecting the input to the hidden layer.

What values of x_i will cause h_i to be maximum (or maximally activated)?

Suppose we assume that our inputs are normalized such that

$$\|x_i\| = 1.$$

So, we are interested to solve the following:

$$\max_{x_i} w_i^T x_i$$

subject to

$$\|x_i\|^2 = x_i^T x_i = 1.$$

The product is maximized when both vectors are in the same direction.

Therefore,

$$x_i = \frac{w_i}{\sqrt{w_i^T w_i}}.$$

Thus, the input which maximally activates the hidden neuron is

$$x_i = \frac{w_i}{\sqrt{w_i^T w_i}}$$

Hence,

$$q_i = \frac{w_1}{\sqrt{w_1^T w_1}}, \quad \frac{w_2}{\sqrt{w_2^T w_2}}, \quad \dots, \quad \frac{w_n}{\sqrt{w_n^T w_n}}.$$

will respectively cause hidden neurons 1 to n to maximally fire.

19.2 Visualization of Learned Features

Which hidden neuron will be learning the different shapes, as we discuss, will be more clear for us with this visualization.

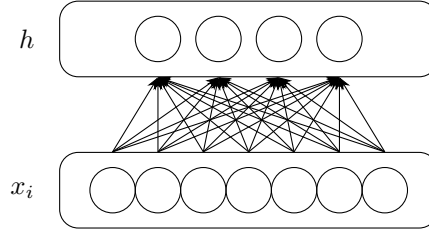


Figure 9: Learned hidden features

We can learn hidden feature values from AE, and feed the feature layer to the classifier.

$$h_i = \sigma(w_i^T x_i).$$

where w_i is the shared vector of weights connecting the input to the hidden layer.

The different values of x_i will cause different hidden neurons to be maximally activated.

Suppose we assume that our inputs are normalized so that

$$\|x_i\| = 1.$$

We are interested to solve

$$\max_{x_i} \{w_i^T x_i\}$$

subject to

$$\|x_i\|^2 = x_i^T x_i = 1.$$

So,

$$x_i = \frac{w_i}{\sqrt{w_i^T w_i}}.$$

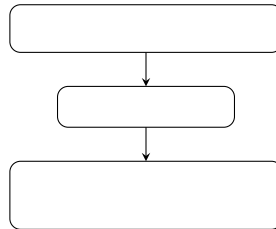
Thus, inputs are

$$q_i = \frac{w_1}{\sqrt{w_1^T w_1}}, \quad \frac{w_2}{\sqrt{w_2^T w_2}}, \quad \dots, \quad \frac{w_n}{\sqrt{w_n^T w_n}}.$$

These will respectively cause hidden neurons 1 to n to maximally fire.

Bias and Variance

- We sample 25 points from training data and train a simple and a complex model.
- We repeat this process 25 times, i.e., train multiple models (each made from a different sample of the training data).
- We are trying to learn the parameter values of the training data.
- The parameter value will change, when we use different samples.

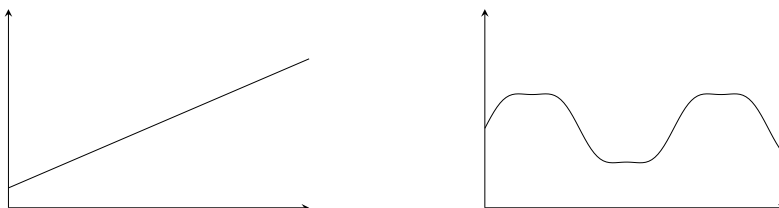


We consider the problem of fitting a curve from a given set of training points.

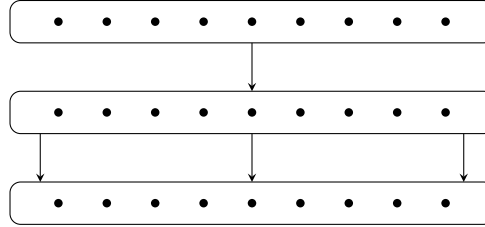
- We consider two models:

$$y = wx + w_0$$

$$y = f(x) = \sum_{i=1}^n w_i x^i + w_0$$

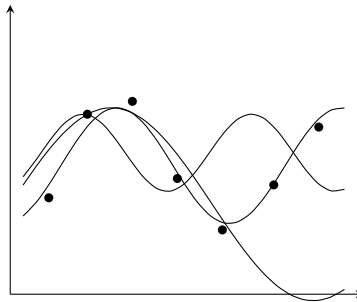


- We sample 25 points from training data and train a simple and a complex model.
- We repeat this process n times.
- We train multiple models (each made from a different sample of the training data).
- We are trying to learn the parameter value of the training data.



(high variance)

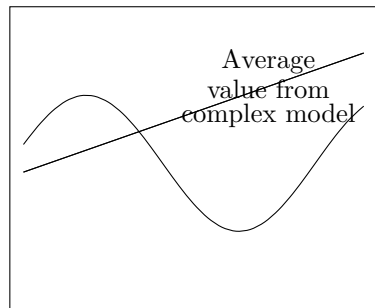
- What is happening here, due high no. of parameters, there is a tendency to overfit, so when we change the sample data points the output function are very different from each other.



Now let us derive few concepts from statistics.

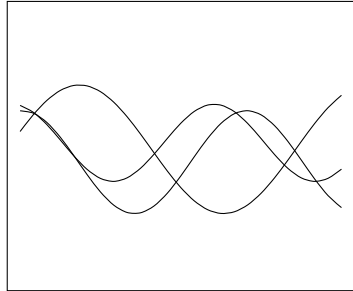
- Let $f(n)$ be the true model (sinusoidal in this case) and $\hat{f}(n)$ be our estimate of the model (simple or complex, in this case) then

$$\text{Bias}(\hat{f}(n)) = E[\hat{f}(n)] - f(n)$$



- Now, if we consider the straight line then we can say that our bias is very high.
- $E[\hat{f}(n)]$ is the average (or expected) value of the model.
- We can see that for the simple model the average value (green line) is very far from the true value $f(n)$ (sinusoidal function).

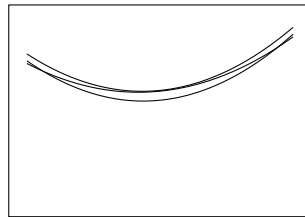
- Mathematically, a simple model has a high bias.
- What is happening here, due high no. of parameters, there is a tendency to overfit, so when we change the sample data points the output functions are very different from each other.



We now define variance:

$$\text{Variance}(\hat{f}(x)) = E \left[\left(\hat{f}(x) - E[\hat{f}(x)] \right)^2 \right]$$

- Roughly speaking, it tells us how much the difference of $\hat{f}(x)$ is determined on different samples of the data differ from each other.
- It is clear that the simple model has a low variance whereas the complex model has a high variance.



- Simple model: high bias, low variance
- Complex model: low bias, high variance
- There is always a trade-off between bias and variance.
- Both bias and variance contribute to the overall prediction error.

Train error VS. Test error

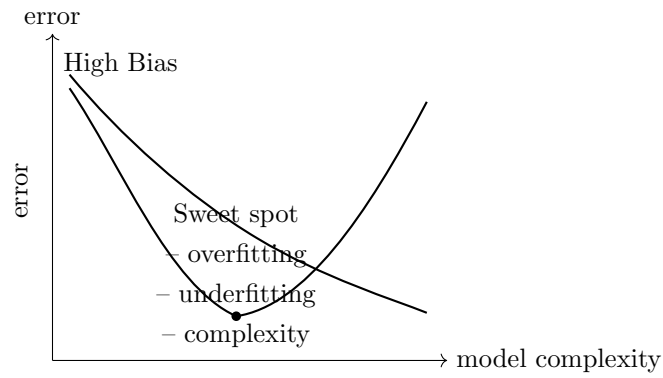
- Consider a new point (x, y) which was not seen during training.
- If we use the model $\hat{f}(x)$ to predict the value of y , then the mean square error is given by

$$\mathbb{E} \left[(y - \hat{f}(x))^2 \right].$$

Every time we get a new training point, hence it can be considered as random.

We can show that

$$\mathbb{E} \left[(y - \hat{f}(x))^2 \right] = \text{Bias}^2 + \text{Variance} + \sigma^2 \quad (\text{irreducible error})$$



- The parameters of $\hat{f}(x)$ (all w_i 's) are trained using a training set

$$\{(x_i, y_i)\}_{i=1}^n.$$

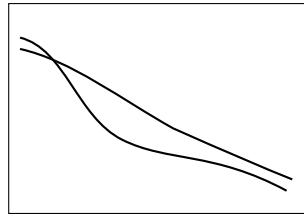
- However, at test time we are interested in evaluating the model on a validation (unseen) set which has not been used for training.
- This gives rise to the following two entities of interest:
 - train error (say, mean square error)
 - test error

- Let there be n training points and m test (validation) points.

$$\text{Train}_{err} = \frac{1}{n} \sum_{i=1}^n \left(y_i - \hat{f}(x_i) \right)^2$$

$$\text{Test}_{err} = \frac{1}{m} \sum_{i=n+1}^{n+m} \left(y_i - \hat{f}(x_i) \right)^2$$

- As model complexity increases, train error becomes overly optimistic and gives us a wrong picture of how close $\hat{f}$ is to f .
- The validation error gives the real picture of how $\hat{f}$ is to f .



- Let

$$D = \{(x_i, y_i)\}_{i=1}^{n+m}.$$

Then for any point (x_i, y_i) we have

$$y_i = f(x_i) + \epsilon_i.$$

- which means that y_i is related to x_i by some same function.
- Both have the same f but there is also same noise ϵ in the regression.
- For simplicity, we assume

$$\epsilon \sim \mathcal{N}(0, \sigma^2).$$

- Further, we use $\hat{f}$ to approximate f and estimate the parameter using T.C.D such that

$$\hat{y}_i = \hat{f}(x_i).$$

We interested in knowing

$$\mathbb{E} \left[\left(\hat{f}(x_i) - f(x_i) \right)^2 \right]$$

but we cannot estimate this directly because we do not know f .

- We will see how to estimate this empirically using the observation $\hat{y}_i$ to prediction $\hat{y}_i$.

Since

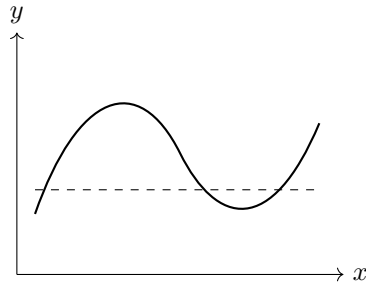
$$y_i = f(x_i) + \epsilon_i,$$

we have

$$\begin{aligned} \mathbb{E} [(\hat{y}_i - y_i)^2] &= \mathbb{E} \left[\left(\hat{f}(x_i) - f(x_i) - \epsilon_i \right)^2 \right] \\ &= \mathbb{E} \left[\left(\hat{f}(x_i) - f(x_i) \right)^2 - 2\epsilon_i \left(\hat{f}(x_i) - f(x_i) \right) + \epsilon_i^2 \right] \\ &= \mathbb{E} \left[\left(\hat{f}(x_i) - f(x_i) \right)^2 \right] \\ &\quad - 2\mathbb{E} \left[\epsilon_i \left(\hat{f}(x_i) - f(x_i) \right) \right] + \mathbb{E} [\epsilon_i^2]. \end{aligned}$$

Therefore,

$$\mathbb{E} \left[\left(\hat{f}(x_i) - f(x_i) \right)^2 \right] = \mathbb{E} [(\hat{y}_i - y_i)^2] - \mathbb{E}[\epsilon_i^2] + 2\mathbb{E} \left[\epsilon_i \left(\hat{f}(x_i) - f(x_i) \right) \right] \quad (1)$$



$$\mathbb{E}[(\hat{y}_i - y_i)^2] = \frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2$$

Empirical because, min is the only way we can do.

case 1: Using test observation

$$\begin{aligned} & \mathbb{E} \left[\left(\hat{f}(x_i) - f(x_i) \right)^2 \right]_{\text{true error}} \\ &= \frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2 - \frac{1}{m} \sum_{i=n+1}^{n+m} \epsilon_i^2 \\ & \quad + 2\mathbb{E} \left[\epsilon_i \left(\hat{f}(x_i) - f(x_i) \right) \right]. \\ & \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} (\hat{y}_i - y_i)^2}_{\text{empirical estimate of error}} \quad - \quad \underbrace{\frac{1}{m} \sum_{i=n+1}^{n+m} \epsilon_i^2}_{\text{small error}} \end{aligned}$$

$$+ 2\mathbb{E} \left[\epsilon_i \left(\hat{f}(x_i) - f(x_i) \right) \right]$$

- ϵ_i is independent of other random variable we see in the expectation.
- None of the test observation participated in the estimation of $\hat{f}(x)$.

$$\begin{aligned} \therefore \quad \epsilon_i &\perp \left(\hat{f}(x_i) - f(x_i) \right) \\ \therefore \quad \mathbb{E} \left[\epsilon_i \left(\hat{f}(x_i) - f(x_i) \right) \right] &= \mathbb{E}[\epsilon_i] \mathbb{E} \left[\hat{f}(x_i) - f(x_i) \right] \\ &= 0 \cdot \mathbb{E} \left[\hat{f}(x_i) - f(x_i) \right] = 0 \end{aligned}$$

$$\boxed{\text{true error} = \text{empirical test error} + \text{small correction}}$$

20 Training Error, Test Error, and Model Complexity

20.1 Training Error vs. Test Error

Consider a new point (x, y) which has not been seen during training. If we use the model $\hat{f}(x)$ to predict the value of y , the mean squared error is

$$\mathbb{E} \left[(y - \hat{f}(x))^2 \right].$$

Every time we add a new training point, the model is retrained. Hence, the training points can be regarded as random observations.

Let the true relationship between x and y be

$$y = f(x) + \epsilon,$$

where ϵ is a random noise term with

$$\mathbb{E}[\epsilon] = 0, \quad \text{Var}(\epsilon) = \sigma^2.$$

The expected squared prediction error can be decomposed as

$$\mathbb{E} \left[(y - \hat{f}(x))^2 \right] = \mathbb{E} \left[\left(f(x) + \epsilon - \hat{f}(x) \right)^2 \right] \quad (1)$$

$$\begin{aligned} &= \mathbb{E} \left[\left(\hat{f}(x) - f(x) \right)^2 \right] + \mathbb{E}[\epsilon^2] \\ &\quad + 2\mathbb{E} \left[\epsilon \left(f(x) - \hat{f}(x) \right) \right]. \end{aligned} \quad (2)$$

Since ϵ is independent of the training data and has zero mean,

$$\mathbb{E} \left[\epsilon \left(\hat{f}(x) - f(x) \right) \right] = \mathbb{E}[\epsilon] \mathbb{E} \left[\hat{f}(x) - f(x) \right] = 0.$$

Therefore,

$$\boxed{\mathbb{E} \left[(y - \hat{f}(x))^2 \right] = \mathbb{E} \left[(\hat{f}(x) - f(x))^2 \right] + \sigma^2} \quad (3)$$

where σ^2 represents the irreducible error.

20.2 Empirical Estimation of Test Error

Suppose the test observations are

$$D_{\text{test}} = \{(x_i, y_i)\}_{i=1}^m,$$

where these observations were not used during training.

The empirical test error can be estimated as

$$\hat{E}_{\text{test}} = \frac{1}{m} \sum_{i=1}^m (y_i - \hat{f}(x_i))^2. \quad (4)$$

This provides an empirical estimate of

$$\mathbb{E} \left[(y - \hat{f}(x))^2 \right].$$

The important point is that the test observations are independent of the observations used for estimating the parameters of $\hat{f}(x)$.

20.3 Case 1: Test Observations

For a test observation,

$$y_i = f(x_i) + \epsilon_i,$$

where ϵ_i is independent of the training data.

Thus,

$$\mathbb{E} \left[\left(\hat{f}(x_i) - f(x_i) \right)^2 \right] = \mathbb{E} [(\hat{y}_i - y_i)^2] - \mathbb{E}[\epsilon_i^2] \quad (5)$$

$$+ 2\mathbb{E} \left[\epsilon_i \left(\hat{f}(x_i) - f(x_i) \right) \right]. \quad (6)$$

Since

$$\mathbb{E}[\epsilon_i] = 0$$

and ϵ_i is independent of $\hat{f}(x_i)$,

$$\mathbb{E} \left[\epsilon_i \left(\hat{f}(x_i) - f(x_i) \right) \right] = 0.$$

Therefore,

$$\boxed{\text{True error} = \text{Empirical test error} - \text{Irreducible error}} \quad (7)$$

up to sampling variation.

20.4 Case 2: Training Observations

Now consider a training observation. The same observations used to estimate the parameters of $\hat{f}(x)$ are also used to calculate the error.

The expected squared error can be written as

$$\mathbb{E} \left[\left(\hat{f}(x_i) - f(x_i) \right)^2 \right] = \frac{1}{n} \sum_{i=1}^n (\hat{y}_i - y_i)^2 - \frac{1}{n} \sum_{i=1}^n \epsilon_i^2 + 2\mathbb{E} \left[\epsilon_i \left(\hat{f}(x_i) - f(x_i) \right) \right]. \quad (8)$$

Unlike the test-error case,

$$\epsilon_i \not\perp \hat{f}(x_i),$$

because the same observation is used for estimating $\hat{f}$.

Consequently,

$$\mathbb{E} \left[\epsilon_i \left(\hat{f}(x_i) - f(x_i) \right) \right] \neq \mathbb{E}[\epsilon_i] \mathbb{E} \left[\hat{f}(x_i) - f(x_i) \right]. \quad (9)$$

Hence, the empirical training error is generally smaller than the true prediction error and does not provide an unbiased estimate of the error on unseen observations.

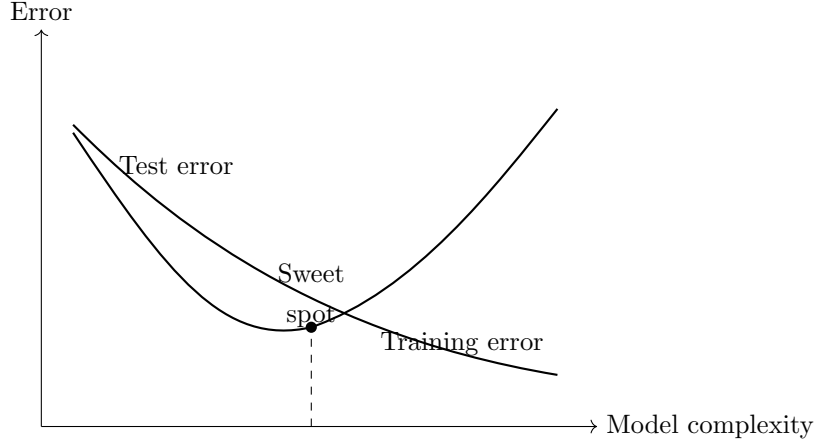
20.5 True Error and Model Complexity

As the complexity of a model increases, the model becomes more flexible and can fit the training data more closely.

For a simple model, the training error is relatively high. As model complexity increases, the training error generally decreases.

However, the test error behaves differently. Initially, increasing model complexity reduces the test error because the model becomes capable of capturing important patterns in the data. After a certain point, excessive complexity causes the model to fit noise in the training data, resulting in overfitting.

Thus, the test error typically has a U-shaped relationship with model complexity.



The point corresponding to the minimum test error represents the desirable level of model complexity. This point is sometimes referred to as the *sweet spot*. The relationship can therefore be summarized as

$$\boxed{\text{True Error} = \text{Empirical Training Error} + \text{Estimation Error} + \Omega(\text{Model Complexity})} \quad (10)$$

where $\Omega(\cdot)$ represents a penalty associated with model complexity.

20.6 Stein's Lemma and Model Complexity

Stein's lemma provides a connection between the sensitivity of the fitted model to the observations and the model's effective complexity.

For observations

$$y_i = f(x_i) + \epsilon_i, \quad \epsilon_i \sim \mathcal{N}(0, \sigma^2),$$

the following quantity is related to the optimism of the training error:

$$\frac{1}{n} \sum_{i=1}^n \mathbb{E} \left[\epsilon_i \left(\hat{f}(x_i) - f(x_i) \right) \right] = \frac{\sigma^2}{n} \sum_{i=1}^n \mathbb{E} \left[\frac{\partial \hat{f}(x_i)}{\partial y_i} \right]. \quad (11)$$

The quantity

$$\boxed{\frac{\partial \hat{f}(x_i)}{\partial y_i}} \quad (12)$$

measures how sensitive the fitted prediction is to a small change in the corresponding observation y_i .

A highly complex model tends to be more sensitive to changes in individual observations, whereas a simple model is generally less sensitive.

20.7 Regularization and Model Complexity

The above discussion provides the basis for regularization methods.

Regularization introduces an additional penalty term into the training objective:

$$\boxed{\min_{\theta} [L(\theta) + \Omega(\theta)]} \quad (13)$$

where

- $L(\theta)$ is the training loss,
- θ represents the model parameters, and
- $\Omega(\theta)$ is the regularization penalty.

The purpose of regularization is to discourage unnecessarily complex models and thereby improve generalization.

For example, in L_2 regularization,

$$\Omega(\theta) = \lambda \sum_j \theta_j^2, \quad (14)$$

while for L_1 regularization,

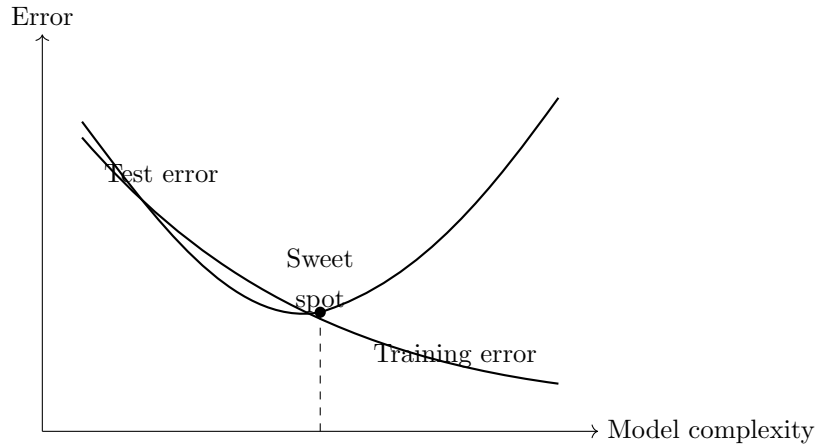
$$\Omega(\theta) = \lambda \sum_j |\theta_j|. \quad (15)$$

Here, λ controls the strength of regularization.

The regularized objective can therefore be written as

$$\theta^* = \arg \min_{\theta} \{L(\theta) + \lambda \Omega(\theta)\}. \quad (16)$$

For complex models, a stronger penalty is generally desirable, whereas simpler models require less regularization.



The regularization penalty should therefore be chosen so that the resulting model achieves an appropriate balance between fitting the training data and controlling model complexity.

In particular,

$$\boxed{\text{Regularization} \implies \text{lower effective model complexity} \implies \text{better generalization.}} \quad (17)$$